

GRENOBLE-INP ENSIMAG

INGÉNIERIE DES SYSTÈMES
D'INFORMATION

2A

GRENOBLE-INP ENSIMAG

INFORMATION SYSTEMS
ENGINEERING

2A



**ÉCOLE NATIONALE SUPÉRIEURE
D'INFORMATIQUE ET DE MATHÉMATIQUES
APPLIQUÉES DE GRENOBLE**

PROJET BD

DOCUMENTATION DU PROJET

Réalisé par :

ARDAN FATIMA-AZZAHRA
DIAKITE Alpha OUSMANE
MENGOSSE ADRIEN
NAMATY YASSER
OUDRHIRI IDRISSE SAFWANE

Responsable :

COMIGNANI UGO

Année académique : 2025–2026

Table des matières

1	INTRODUCTION	3
2	ANALYSE DU SYSTÈME	4
2.1	Dépendances Fonctionnelles (DF)	4
2.2	Contraintes de Multiplicité (CM)	5
2.3	Contraintes de Valeur (CV)	5
2.4	Contraintes Contextuelles (CT)	5
3	Conception Entité–Association	7
3.1	Principales entités	7
3.2	Associations et cardinalités essentielles	7
3.3	Contraintes métier intégrées dans le modèle	8
3.4	Spécialisations (ISA)	8
3.5	Rôle du modèle E/A	8
3.6	Diagramme Entité–Association	9
4	RELATIONNEL	12
4.1	Traduction du diagramme Entité–Association	12
4.2	Formes normales	19
4.3	Contraintes applicatives non implantables	19
5	FONCTIONNALITÉS	21
5.1	Fonctionnalité 1	21
5.2	Fonctionnalité 2 : Alertes de péremption et réduction des prix . . .	24
5.3	Fonctionnalité 3 : Clôture d’une commande (retrait ou livraison) . .	27
6	BILAN DU PROJET	30
6.1	Organisation de l’Équipe et Déroulement	30
6.2	Outils et Choix Techniques	30
6.3	Difficultés Rencontrées	30
7	GUIDE D’UTILISATION@	32
7.1	Lancer l’application :	32
7.2	Fenêtre de sélection du rôle :	32
7.3	Login	32

7.4	Boutique : Epicerie au Valais	32
7.5	Catalogue des produits :	33
7.6	Passer une commande :	34
7.7	Alertes de Prémption :	37
7.8	Clôturer la commande :	37
8	CONCLUSION	40



1 INTRODUCTION

Ce document présente le bilan du projet de Système de Gestion de Bases de Données Relationnelles pour l'épicerie locale "Au Valaisan". Notre objectif principal était de concevoir et de mettre en œuvre la base de données de l'épicerie ainsi que l'application d'interaction associée, en mettant l'accent sur la gestion des produits, des stocks, des clients et des commandes.

Le projet a été réalisé par notre équipe, en utilisant le langage Java et l'API JDBC pour le développement de l'application. Une attention particulière a été portée à l'implémentation des contraintes métiers spécifiques à la vente en vrac et de produits locaux, incluant la saisonnalité, la gestion des conditionnements, et la complexité du stock en lots (DLC/DLUO).

Ce rapport synthétise notre approche, notre organisation, le déroulement du projet, les choix techniques effectués, ainsi que les difficultés rencontrées et les succès obtenus au cours de ces trois semaines de travail intense.

2 ANALYSE DU SYSTÈME

2.1 Dépendances Fonctionnelles (DF)

Nous avons identifié les dépendances fonctionnelles clés pour la modélisation du schéma relationnel.

-- Dépendances Fonctionnelles (DF) Clées --

idContact -> nom, numTel, email, prenom

idAdresse -> rue, ville, codePostal

Producteur (id Producteur) -> latitude, longitude

Produit (id Produit) -> nom Produit, catégorie, description, caracteristiques
specifiques, origineGéographique, délaiDisponibilité

Lien Produit/Prod. (id Produit, id Producteur) -> id Stock

Conditionnement (id Conditionnement) -> prix Achat Producteur, prix Vente Client

idConditionnementPréconditionné -> poidsSachet

Contenant (reference Contenant) -> type, capacite, stock, caractere, prix

Lot (id Lot) -> date Reception, date limite, type Date limite (DLC/DLUO)

idLotVrac -> quantitéDisponibleVrac

idLotPréconditionné -> quantitéDisponiblePréconditionné

Perte (idPerte) -> datePerte, naturePerte

idPerteContenant -> quantitéPerdueContenant

idPerteVrac -> quantitéPerdueVrac

idPertePréconditionné -> quantitéPerduePréconditionné

Commande (id Commande) -> dateCommande, heureCommande, statutCommande,
modePaiement, modeRécupération, dateRécupération

idLigneCommandeVrac -> quantitéCommandeVrac

idLigneCommandePréconditionné -> quantitéCommandePréconditionné

idLigneCommandeContenant -> quantitéCommandeContenant

idLigneCommande -> prixUnitaire, sousTotalLigne

idLigneCommandeProduit, idCommande -> idProduit, idProducteur

Mode Recuper. Domicile (id Mode Recuperation Domicile) -> paysLivraison,
poidsTotal, distanceAdresseBoutique, dateEstimeeLivraison, typePaysLivraison

idCommande -> idModeRécupération

2.2 Contraintes de Multiplicité (CM)

Ces contraintes indiquent les liens où une entité peut être associée à plusieurs instances d'une autre (multiplicité N).

- Un **Producteur** peut fournir plusieurs **Produits** et exercer plusieurs **Types d'Activité**.
- Un **Produit** peut être associé à plusieurs **Disponibilités** (saisonnalité) ou aucune et plusieurs **Conditionnements** (Vrac et Pré-conditionné).
- Un **Stock** est divisé en plusieurs **Lots** ou aucun (gestion DLC/DLUO).
- Un **Produit** associé à un **Producteur** peut faire l'objet d'aucune ou plusieurs **Pertes**.
- Une **referenceContenant** détermine aucune ou plusieurs **PerteContenant**.
- Un **Client** peut avoir plusieurs **Adresses** et passer plusieurs **Commandes**.
- Une **Commande** est composée de plusieurs **Lignes de Commande**.

2.3 Contraintes de Valeur (CV)

Les contraintes de valeur spécifient les domaines d'énumération des attributs.

- **typeDateLimite** $\in \{DLC, DLUO\}$.
- **naturePerte** $\in \{vol, Casse\}$.
- **typeContenant** $\in \{bocauxenverrerutilisable, sachetKraft, sachetentissus, papiercir\}$.
- **caractèreContenant** $\in \{reutilisable, jetable\}$.
- **statutCommande** $\in \{EnPrparation, Prte, Enlivraison, Rcupre/Livre, Annul\}$.
- **typePaysLivraison** $\in \{Francmetropolitaine, DOM - TOM, international\}$.
- **statutProduit** $\in \{Disponible, PasDisponible\}$.
- **Mode Paiement** $\in \{Enligne, Boutique\}$.
- **Contraintes d'Extension** : Les ensembles d'identifiants Conditionnement et Lot sont des unions disjointes de leurs sous-types (Ex : $Ext(Conditionnement) = Ext(Cond.Pr - conditionn) \cup Ext(Cond.Vrac)$). Il en est de même pour les Perte, PerteProduit, LigneCommande et LigneCommandeProduit.

2.4 Contraintes Contextuelles (CT)

Règles de gestion spécifiques qui devront être implémentées dans les transactions et l'application.

- **Conditionnement Vrac** : Un seul par produit et par producteur. Plusieurs sont possibles pour le Pré-conditionné.
- **Livraison** : Pour un produit donné, il y a au maximum une livraison par jour

- **Gestion des Lots** : Le système alerte **7 jours avant** DLC/DLUO et peut réduire automatiquement le prix.
- **Droit à l'Oubli** : Anonymisation des données client (sauf historique de commande pour comptabilité).
- **Cycle de Vie de la Commande et Stock** :
 1. Statut "**En Préparation**" : Annulation client possible. Les articles **restent disponibles à la vente** (gestion de la concurrence nécessaire).
 2. Statut "**Prête**" : Les produits **sortent du stock** et l'annulation client est bloquée. Le paiement en boutique s'effectue à ce statut.
 3. Annulation par l'épicerie possible jusqu'au statut "**Expédié**".
- **Disponibilité** : Nécessite une vérification du stock, de la saisonnalité (si hors saison → indisponible) et du délai pour les produits sur commande. Le stock doit être affiché **en temps réel**.

3 Conception Entité–Association

La conception Entité–Association constitue la base du modèle conceptuel du projet. Elle permet de représenter les informations essentielles de l'épicerie, ainsi que les règles métier identifiées dans le cahier des charges.

Les objectifs de cette conception sont :

- identifier les entités nécessaires à la gestion des produits, stocks, commandes et clients ;
- définir les associations et leurs cardinalités ;
- représenter les contraintes métier importantes (péremption, saisonnalité, conditionnements, etc.).

3.1 Principales entités

- **Producteur** : informations d'exploitation, coordonnées, activités (agriculteur, éleveur...).
- **Produit** : nom, catégorie, caractéristiques (bio, allergènes, origine), producteur d'origine.
- **Saison** : périodes annuelles de disponibilité (un produit peut en avoir plusieurs).
- **Stock** : stock global associé à un couple Produit–Producteur.
- **Lot** : quantité, date de réception, date limite (DLC ou DLUO), type de conditionnement.
- **Conditionnement** : vrac ou préconditionné, avec prix d'achat et prix de vente.
- **Contenant** : emballages vendus séparément (bocal, sachet...), avec capacité et type.
- **Client** : identifié par une adresse e-mail ; peut posséder plusieurs adresses.
- **Commande** : regroupement de lignes, avec date, statut et mode de récupération.
- **LigneCommande** : produit ou contenant, quantité, prix unitaire au moment de la commande.
- **Perte** : enregistrement des pertes (vols, casse) avec date, nature et quantité.

3.2 Associations et cardinalités essentielles

- **Producteur – Produit** : (1,N). Un producteur fournit plusieurs produits.
- **Produit – Saison** : (1,N). Permet de gérer les produits saisonniers.
- **Produit – Conditionnement** : (1,N). Vrac et/ou préconditionné.
- **Conditionnement – Contenant** : (N,N). Seulement pour le vrac.
- **Stock – Lot** : (1,N). La gestion de la péremption se fait au niveau du lot.
- **Client – Adresse** : (1,N).

- **Commande – LigneCommande** : (1,N).
- **LigneCommande – Produit/Contenant** : (1,1).

3.3 Contraintes métier intégrées dans le modèle

- **DLC / DLUO mutuellement exclusives** : un lot possède une seule date limite.
- **Péréemption gérée au niveau du lot** : indispensable pour la fonctionnalité d'alertes.
- **Saisonnalité** : un produit hors saison ne peut pas être commandé.
- **Stock et commandes** : le stock n'est déduit qu'au passage au statut *prête*.
- **Conditionnements multiples** : vrac et préconditionné ont des attributs différents.
- **Droit à l'oubli** : anonymisation d'un client tout en conservant l'historique des commandes.
- **Gestion des pertes** : table dédiée pour la casse ou le vol d'un lot.

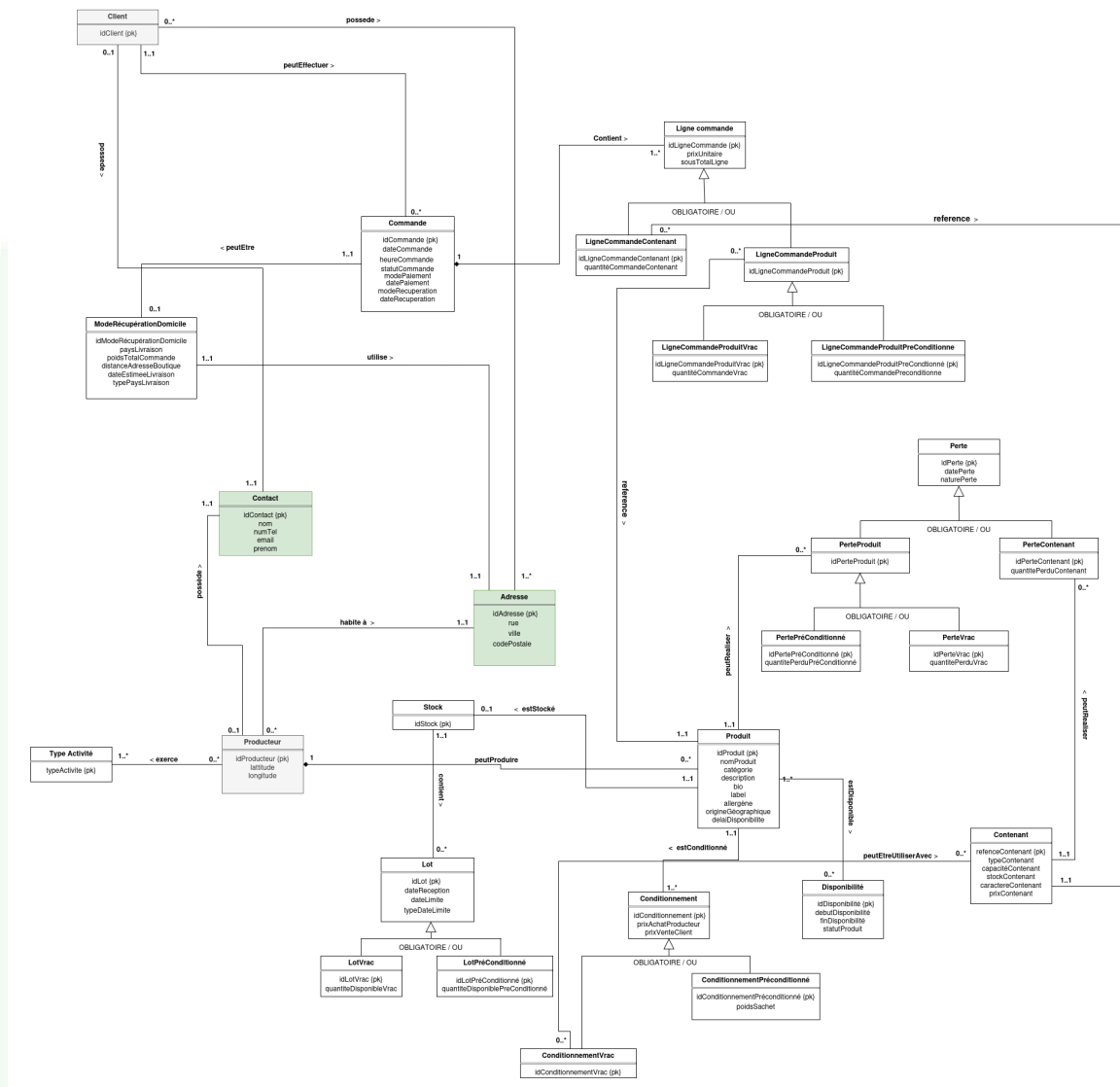
3.4 Spécialisations (ISA)

- **Conditionnement**
 - *Vrac*
 - *Préconditionné*
- **ModeRécupération**
 - *Boutique*
 - *Livraison*
- **LigneCommande**
 - *LigneProduit*
 - *LigneContenant*

3.5 Rôle du modèle E/A

Ce modèle a permis de :

- structurer l'ensemble des données avant la traduction relationnelle ;
- valider les règles métier complexes (péréemption, saisonnalité, gestion des commandes) ;
- identifier les associations nécessaires et éviter les redondances ;
- garantir une base relationnelle cohérente et facilement exploitable.



Choix de modélisation

Gestion des deux modes de conditionnement (vrac/pré-conditionné) On a choisi une spécialisation par table pour distinguer les conditionnements vrac et pré-conditionnés. Chaque conditionnement hérite de la table `Conditionnement` et est soit dans `ConditionnementVrac`, soit dans `ConditionnementPreconditionne` (exclusivité mutuelle). Cette approche permet de gérer des attributs spécifiques : le préconditionné a un `poidsSachet`, tandis que le vrac peut être associé à des contenants via l'association `PeutEtreUtiliseAvec`. De même, les lots sont spécialisés en `LotVrac` et `LotPreconditionne`, et les lignes de commande en `LigneCommandeProduitVrac` et `LigneCommandeProduitPreconditionne`. Cette modélisation garantit la cohérence des

types et facilite la gestion des quantités (en kilos pour le vrac, en unités pour le préconditionné).

Modélisation de la saisonnalité La saisonnalité est gérée via la table `Disponibilite` qui définit des périodes avec `debutDisponibilite` et `finDisponibilite`. L'association `ProduitEstDisponible` lie un produit à plusieurs périodes de disponibilité. Lors de la vérification de disponibilité, on teste si la date courante est dans une période active. Cette modélisation permet de gérer des produits avec plusieurs saisons dans l'année et d'indiquer au client la prochaine période de disponibilité si le produit est hors saison.

Distinction entre produits et contenants Les contenants sont modélisés comme une entité séparée (`Contenant`) car ils ont leurs propres caractéristiques : type, capacité, stock, caractère réutilisable/non-réutilisable, et prix. Ils sont associés aux conditionnements vrac via `PeutEtreUtiliseAvec`, car seuls les produits en vrac nécessitent un contenant. Dans les commandes, les contenants sont gérés via des lignes distinctes (`LigneCommandeContenant`) séparées des lignes produits (`LigneCommandeProduit`), ce qui permet de commander des contenants seuls ou d'associer plusieurs contenants à une commande.

Gestion des pertes (vols et casse) Les pertes sont modélisées par la table `Perte` avec un attribut `naturePerte` (Vol ou Casse). Les pertes de produits sont spécialisées en `PerteProduit` (liée à un lot), puis en `PerteProduitVrac` ou `PerteProduitPreconditionne` selon le type. Les pertes de contenants sont gérées via `PerteContenant`. Cette modélisation permet de tracer l'origine des pertes (lot concerné) et leur nature, tout en respectant la distinction vrac/préconditionné pour les quantités perdues.

Mise en œuvre du droit à l'oubli Le droit à l'oubli est géré via les contraintes `ON DELETE CASCADE` sur les clés étrangères liées au `Client`. Lors de la suppression d'un client, ses données personnelles (`Contact`, `Adresse` via `PossedeAdresse`) sont automatiquement supprimées. Cependant, l'historique des commandes doit être conservé pour des raisons comptables et légales. On a donc choisi de ne pas mettre de `ON DELETE CASCADE` sur `Commande(idClient)`, permettant de conserver les commandes tout en anonymisant les données personnelles du client (on pourra changer son `idClient` pour plus d'anonymisation).

Stratégie de gestion du stock pendant le cycle de vie d'une commande On a implémenté une stratégie où le stock n'est pas décrémenté à la création de la commande,

mais uniquement au passage au statut “Prête”. Pour les commandes **Domicile**, le stock est sorti lors de la préparation (passage à “Prête”) car le paiement est effectué en ligne et la commande est déjà validée. Pour les commandes **Boutique**, le stock peut être sorti soit lors du passage à “Prête”, soit à la clôture (récupération), selon la stratégie choisie. La sortie de stock utilise la stratégie FEFO (First-Expired, First-Out) : les lots sont triés par date limite croissante et les quantités sont retirées en priorité des lots les plus proches de la péremption, minimisant ainsi les pertes.



4 RELATIONNEL

4.1 Traduction du diagramme Entité-Association

On a traduit notre diagramme Entité-Association en un schéma relationnel composé de 25 tables. On présente ci-dessous la traduction littérale de chaque table avec ses attributs, clés primaires, clés étrangères et contraintes.

Entités simples

- **Contact**(idContact, nom, prénom, numTel, email)
 - Contraintes : nom NOT NULL, prénom NOT NULL
 - **Contraintes non implantables** :
 - Un contact est soit celui d'un producteur, soit celui d'un client (exclusivité mutuelle).
- **Client**(idClient, idContact)
 - Clé étrangère : idContact référence Contact(idContact) ON DELETE CASCADE
 - Contraintes : idContact NOT NULL UNIQUE
 - **Contraintes non implantables** :
 - Droit à l'oubli : gestion particulière pour supprimer les données tout en conservant l'historique des commandes si nécessaire.
- **Adresse**(idAdresse, rue, ville, codePostal)
 - Contraintes : rue NOT NULL, ville NOT NULL, codePostal NOT NULL
 - **Contraintes non implantables** :
 - Une adresse est soit celle d'un producteur, soit celle d'un client (via PossedeAdresse).
- **Producteur**(idProducteur, latitude, longitude, idAdresse, idContact)
 - Clé étrangère : idAdresse référence Adresse(idAdresse)
 - Clé étrangère : idContact référence Contact(idContact) ON DELETE CASCADE idContact NOT NULL UNIQUE
- **TypeActivite**(typeActivite)
- **Contenant**(referenceContenant, typeContenant, capaciteContenant, stockContenant, caractereContenant, prixContenant)
 - Contraintes : typeContenant NOT NULL, capaciteContenant NOT NULL, stockContenant NOT NULL, caractereContenant NOT NULL, prixContenant NOT NULL, capaciteContenant > 0, stockContenant >= 0, caractereContenant IN ('Réutilisable', 'Non-Réutilisable'), prixContenant >= 0

- **Disponibilite**(idDisponibilite, debutDisponibilite, finDisponibilite, statutProduit)
 - Contraintes : debutDisponibilite NOT NULL, finDisponibilite NOT NULL, statutProduit NOT NULL ,statutProduit IN ('Disponible', 'Pas disponible'), finDisponibilite >= debutDisponibilite
- **Perte**(idPerte, datePerte, naturePerte)
 - Contraintes : datePerte NOT NULL, naturePerte NOT NULL ,naturePerte IN ('Vol', 'Casse')
- **Produit**(idProduit, idProducteur, categorie, description, bio, label, nomProduit, allergene, origineGeographique, delaiDisponibilite)
 - Clé étrangère : idProducteur référence Producteur(idProducteur) ON DELETE CASCADE
 - Contraintes : categorie NOT NULL, nomProduit NOT NULL
- **Stock**(idStock, idProduit, idProducteur)
 - Clé étrangère : (idProduit, idProducteur) référence Produit(idProduit, idProducteur) ON DELETE CASCADE
 - Contraintes : idProduit NOT NULL, idProducteur NOT NULL
- **Commande**(idCommande, dateCommande, dateRecuperation, datePaiement, heureCommande, statutCommande, modePaiement, modeRecuperation, idClient)
 - Clé étrangère : idClient référence Client(idClient) ON DELETE CASCADE
 - Contraintes : dateCommande NOT NULL, heureCommande NOT NULL, statutCommande NOT NULL, modePaiement NOT NULL, modeRecuperation NOT NULL, idClient NOT NULL ,statutCommande IN ('En préparation', 'Prête', 'En livraison', 'Annulée', 'Récupérée/Livrée'), modePaiement IN ('En ligne', 'En Boutique'), modeRecuperation IN ('Boutique', 'Domicile')
 - **Contraintes non implantables :**
 - Annulation par le client autorisée tant que la commande est "En préparation".
 - Paiement en boutique uniquement si le statut est "Prête".
 - Pas de statut "En préparation" pour une commande avec le mode de récupération "Boutique".
 - dateRecuperation et datePaiement sont non NULL si statutCommande = 'Récupérée/Livrée'.
 - Pas de possibilité de payer en boutique si le mode de récupération est "Domicile".

- `dateRecuperation` $\geq$ `dateCommande`.
- `datePaielement` $\leq$ `dateRecuperation` (si les deux sont non NULL).
- **ModeRecuperationDomicile**(idModeRecuperationDomicile, paysLivraison, poidsTotalCommande, distanceAdresseBoutique, dateEstimeeLivraison, typePaysLivraison, idCommande, idAdresse)
 - Clé étrangère : `idCommande` référence `Commande(idCommande)` ON DELETE CASCADE
 - Clé étrangère : `idAdresse` référence `Adresse(idAdresse)`
 - Contraintes : `paysLivraison` NOT NULL, `poidsTotalCommande` NOT NULL, `distanceAdresseBoutique` NOT NULL, `typePaysLivraison` NOT NULL, `idCommande` NOT NULL UNIQUE, `idAdresse` NOT NULL, `poidsTotalCommande` > 0 , `distanceAdresseBoutique` ≥ 0 , `typePaysLivraison` IN ('France Métropolitaine', 'DOM-TOM', 'International')
 - **Contraintes non implantables** :
 - La date de livraison (`dateEstimeeLivraison`) doit être communiquée au client.
 - `dateEstimeeLivraison` $\geq$ `dateCommande` (de la commande associée).
- **LigneCommande**(idLigneCommande, idCommande, prixUnitaire, sousTotalLigne)
 - Clé étrangère : `idCommande` référence `Commande(idCommande)` ON DELETE CASCADE
 - Contraintes : `prixUnitaire` NOT NULL, `sousTotalLigne` NOT NULL, `idCommande` NOT NULL, `prixUnitaire` ≥ 0 , `sousTotalLigne` ≥ 0
 - **Contraintes non implantables** :
 - Le total de la commande doit être égal à la somme des `sousTotalLigne` de toutes les lignes.

Sous-types d'entités

- **Lot**(idLot, dateReception, dateLimite, typeDateLimite, idStock)
 - Clé étrangère : `idStock` référence `Stock(idStock)` ON DELETE CASCADE
 - Contraintes : `dateReception` NOT NULL, `dateLimite` NOT NULL, `typeDateLimite` NOT NULL, `typeDateLimite` IN ('DLC', 'DLUO')
 - **Contraintes non implantables** :
 - `dateLimite` $\geq$ `dateReception`.
 - Un lot correspond à une livraison à l'épicerie d'un jour particulier.
- **LotVrac**(idLot, quantiteDisponibleVrac)
 - Clé étrangère : `idLot` référence `Lot(idLot)` ON DELETE CASCADE

- Contraintes : `quantiteDisponibleVrac NOT NULL ,quantiteDisponibleVrac >= 0`
- **Contraintes non implantables :**
 - Un lot ne peut être à la fois vrac ET préconditionné (exclusivité mutuelle).
 - La quantité disponible doit être cohérente avec les quantités commandées et les pertes.
- **LotPreconditionne**(idLot, quantiteDisponiblePreconditionne)
 - Clé étrangère : `idLot` référence `Lot(idLot)` ON DELETE CASCADE
 - Contraintes : `quantiteDisponiblePreconditionne NOT NULL ,quantiteDisponiblePreconditionne >= 0`
 - **Contraintes non implantables :**
 - Un lot ne peut être à la fois vrac ET préconditionné (exclusivité mutuelle).
 - La quantité disponible doit être cohérente avec les quantités commandées et les pertes.
- **Conditionnement**(idConditionnement, prixAchatProducteur, prixVenteClient, idProduit, idProducteur)
 - Clé étrangère : (`idProduit`, `idProducteur`) référence `Produit(idProduit, idProducteur)` ON DELETE CASCADE
 - Contraintes : `prixAchatProducteur NOT NULL, prixVenteClient NOT NULL ,prixAchatProducteur >= 0, prixVenteClient >= 0`
- **ConditionnementPreconditionne**(idConditionnement, poidsSachet)
 - Clé étrangère : `idConditionnement` référence `Conditionnement(idConditionnement)` ON DELETE CASCADE
 - Contraintes : `poidsSachet NOT NULL ,poidsSachet > 0`
- **ConditionnementVrac**(idConditionnement)
 - Clé étrangère : `idConditionnement` référence `Conditionnement(idConditionnement)` ON DELETE CASCADE
- **LigneCommandeContenant**(idLigneCommande, idCommande, referenceContenant, quantiteCommandeContenant)
 - Clé étrangère : (`idLigneCommande`, `idCommande`) référence `LigneCommande(idLigneCommande, idCommande)` ON DELETE CASCADE, `referenceContenant` référence `Contenant(referenceContenant)` ON DELETE CASCADE
 - Contraintes : `quantiteCommandeContenant NOT NULL ,quantiteCommandeContenant > 0`
 - **Contraintes non implantables :**
 - Une ligne de commande ne peut être à la fois produit ET contenant (exclu-

sivité mutuelle).

- **LigneCommandeProduit**(idLigneCommande, idCommande, idProduit, idProducteur)
 - Clé étrangère : (idLigneCommande, idCommande) référence LigneCommande(idLigneCommande, idCommande) ON DELETE CASCADE
 - Clé étrangère : (idProduit, idProducteur) référence Produit(idProduit, idProducteur)
 - Contraintes : idProduit NOT NULL, idProducteur NOT NULL
 - **Contraintes non implantables :**
 - Une ligne de commande ne peut être à la fois produit ET contenant (exclusivité mutuelle).
- **LigneCommandeProduitVrac**(idLigneCommande, idCommande, quantiteCommandeVrac)
 - Clé étrangère : (idLigneCommande, idCommande) référence LigneCommandeProduit(idLigneCommande, idCommande) ON DELETE CASCADE
 - Contraintes : quantiteCommandeVrac NOT NULL, quantiteCommandeVrac > 0
 - **Contraintes non implantables :**
 - Une ligne de commande produit ne peut être à la fois vrac ET préconditionné (exclusivité mutuelle).
 - La quantité commandée doit être inférieure ou égale à la quantité disponible dans le lot.
- **LigneCommandeProduitPreconditionne**(idLigneCommande, idCommande, quantiteCommandePreconditionne)
 - Clé étrangère : (idLigneCommande, idCommande) référence LigneCommandeProduit(idLigneCommande, idCommande) ON DELETE CASCADE
 - Contraintes : quantiteCommandePreconditionne NOT NULL, quantiteCommandePreconditionne > 0
 - **Contraintes non implantables :**
 - Une ligne de commande produit ne peut être à la fois vrac ET préconditionné (exclusivité mutuelle).
 - La quantité commandée doit être inférieure ou égale à la quantité disponible dans le lot.
- **PerteProduit**(idPerte, idProduit, idProducteur)
 - Clé étrangère : idPerte référence Perte(idPerte) ON DELETE CASCADE
 - Clé étrangère : (idProduit, idProducteur) référence Produit(idProduit,

- idProducteur)
- Contraintes : idProduit NOT NULL, idProducteur NOT NULL
- **Contraintes non implantables :**
 - Une perte ne peut être à la fois produit ET contenant (exclusivité mutuelle).
- **PerteProduitPreconditionne**(idPerte, quantitePerduePreconditionne)
 - Clé étrangère : idPerte référence PerteProduit(idPerte) ON DELETE CASCADE
 - Contraintes : quantitePerduePreconditionne NOT NULL ,quantitePerduePreconditionne > 0
 - **Contraintes non implantables :**
 - Une perte produit ne peut être à la fois vrac ET préconditionné (exclusivité mutuelle).
- **PerteProduitVrac**(idPerte, quantitePerdueVrac)
 - Clé étrangère : idPerte référence PerteProduit(idPerte) ON DELETE CASCADE
 - Contraintes : quantitePerdueVrac NOT NULL ,quantitePerdueVrac > 0
 - **Contraintes non implantables :**
 - Une perte produit ne peut être à la fois vrac ET préconditionné (exclusivité mutuelle).
- **PerteContenant**(idPerte, quantitePerdueContenant, referenceContenant)
 - Clé étrangère : idPerte référence Perte(idPerte) ON DELETE CASCADE
 - Clé étrangère : referenceContenant référence Contenant(referenceContenant)
 - Contraintes : quantitePerdueContenant NOT NULL, referenceContenant NOT NULL ,quantitePerdueContenant > 0
 - **Contraintes non implantables :**
 - Une perte ne peut être à la fois produit ET contenant (exclusivité mutuelle).

Associations

- **PossedeAdresse**(idClient, idAdresse)
 - Clé étrangère : idClient référence Client(idClient) ON DELETE CASCADE
 - Clé étrangère : idAdresse référence Adresse(idAdresse) ON DELETE CASCADE
- **Exerce**(idProducteur, typeActivite)
 - Clé étrangère : idProducteur référence Producteur(idProducteur) ON DELETE CASCADE

- Clé étrangère : `typeActivite` référence `TypeActivite(typeActivite)` ON DELETE CASCADE
- **ProduitEstDisponible**(`idProduit`, `idProducteur`, `idDisponibilite`)
 - Clé étrangère : (`idProduit`, `idProducteur`) référence `Produit(idProduit, idProducteur)` ON DELETE CASCADE
 - Clé étrangère : `idDisponibilite` référence `Disponibilite(idDisponibilite)` ON DELETE CASCADE
- **PeutEtreUtiliseAvec**(`idConditionnement`, `referenceContenant`)
 - Clé étrangère : `idConditionnement` référence `ConditionnementVrac(idConditionnement)` ON DELETE CASCADE
 - Clé étrangère : `referenceContenant` référence `Contenant(referenceContenant)` ON DELETE CASCADE

Justification des choix pour les sous-types d'entités

On a utilisé la technique de spécialisation par table pour gérer les relations d'héritage. Cette approche consiste à créer une table pour la classe mère et des tables séparées pour chaque sous-classe, avec une clé étrangère de la sous-classe vers la classe mère.

Lot → **LotVrac** / **LotPreconditionne** On a choisi cette modélisation car les attributs spécifiques sont différents (`quantiteDisponibleVrac` FLOAT vs `quantiteDisponiblePreconditionne` INTEGER), les types de données diffèrent, et un lot ne peut être qu'en vrac OU préconditionné (exclusivité mutuelle). La table **Lot** contient les attributs communs, et les tables spécialisées héritent via `idLot` qui est à la fois leur clé primaire et une clé étrangère vers **Lot**.

Conditionnement → **ConditionnementVrac** / **ConditionnementPreconditionne** Même principe : **Conditionnement** contient les attributs communs, **ConditionnementPreconditionne** ajoute `poidsSachet`, et **ConditionnementVrac** permet de distinguer les conditionnements vrac pour les lier avec des contenants. Un conditionnement ne peut être qu'en vrac OU préconditionné (exclusivité mutuelle).

LigneCommande → **LigneCommandeProduit** / **LigneCommandeContenant**
 → **LigneCommandeProduitVrac** / **LigneCommandeProduitPreconditionne**
 On a créé une hiérarchie à deux niveaux : **LigneCommande** est spécialisée en **LigneCommandeProduit** (pour les produits) et **LigneCommandeContenant** (pour les contenants), puis **LigneCommandeProduit** est spécialisée en **LigneCommandeProduitVrac** et

LigneCommandeProduitPreconditionne. Cette modélisation permet de gérer les différences d'attributs et de traitements selon le type de ligne.

Perte → **PerteProduit** / **PerteContenant** → **PerteProduitVrac** / **PerteProduitPreconditionne** Même principe appliqué pour distinguer les pertes de produits (qui nécessitent une référence au produit) des pertes de contenants (qui nécessitent une référence au contenant).

4.2 Formes normales

Troisième forme normale (3NF)

On a vérifié que nos tables respectent la 3NF. Les attributs non-clés dépendent directement de la clé primaire, sans dépendances transitives problématiques. On note que pour **Adresse**, on a choisi de ne pas modéliser la dépendance `codePostal` → `ville` car un code postal peut correspondre à plusieurs villes et on veut permettre la flexibilité pour les adresses internationales.

Forme normale de Boyce-Codd (FNBCCK)

La plupart de nos tables sont presque en FNBCCK. Les cas particuliers sont **Adresse** (dépendance `codePostal` → `ville` non modélisée pour flexibilité) et **ModeRecuperationDomicile** (dépendance `paysLivraison` → `typePaysLivraison` conservée pour des raisons pratiques).

4.3 Contraintes applicatives non implantables

Contraintes temporelles

- `dateRecuperation` ≥ `dateCommande`
- `datePaiement` ≤ `dateRecuperation` (si les deux sont non NULL)
- `dateLimite` ≥ `dateReception` (pour Lot)
- `dateEstimeeLivraison` ≥ `dateCommande` (pour ModeRecuperationDomicile)

Contraintes métier complexes

- Annulation par le client autorisée tant que la commande est "En préparation"
- Paiement en boutique uniquement si le statut est "Prête"
- Pas de statut "En préparation" pour une commande avec le mode de récupération "Boutique"

- `dateRecuperation` et `datePaielement` sont non NULL si `statutCommande = 'Récupérée/Livrée'`
- Pas de possibilité de payer en boutique si le mode de récupération est "Domicile"
- Un contact est soit celui d'un producteur, soit celui d'un client (exclusivité mutuelle)
- Une adresse est soit celle d'un producteur, soit celle d'un client
- Un lot correspond à une livraison à l'épicerie d'un jour particulier
- Un seul conditionnement vrac pour un même produit et producteur
- La date de livraison doit être communiquée au client

Contraintes de cohérence transactionnelle

- Stock suffisant avant de valider une commande
- Cohérence des quantités : `quantiteCommandeVrac` $\leq$ `quantiteDisponibleVrac` du lot correspondant
- Total de la commande = somme des `sousTotalLigne`
- Quantité disponible cohérente avec les quantités commandées et les pertes

Gestion de la disponibilité

La disponibilité d'un produit dépend de plusieurs facteurs :

- Pour les produits en stock : disponibilité immédiate si quantité suffisante, indisponible si hors saison avec indication de la prochaine période
- Pour les produits sur commande : délai de disponibilité spécifié par produit (ex : 72h, 1 semaine), client informé du délai
- Le nombre de produits disponibles en stock doit être calculé en tenant compte des quantités commandées et des pertes

Contraintes d'intégrité référentielle supplémentaires

- `ModeRecuperationDomicile` uniquement si `modeRecuperation = "Domicile"`
- Conservation de l'historique des commandes passées même si les produits sont supprimés
- Gestion des suppressions en cascade selon les contraintes métier (ex : annuler plutôt que supprimer les commandes en cours)

5 FONCTIONNALITÉS

5.1 Fonctionnalité 1

Cette section présente la mise en place de la transaction SQL permettant de garantir la cohérence lors du passage d'une commande par un client. L'objectif principal est d'assurer que toutes les lignes de commande soient correctement créées, que les calculs associés sont exacts, et que le stock est vérifié de manière fiable grâce à l'utilisation du niveau d'isolation *Repeatable Read*.

La méthode `validerCommande()` implémente l'ensemble de cette logique transactionnelle et constitue le cœur de ce travail.

Déroulement de la transaction

La méthode `validerCommande()` suit la séquence suivante :

Initialisation Création des services et DAO nécessaires, puis ouverture de la connexion en mode transactionnel.

Traitement de chaque élément de commande Pour chaque élément de la liste `listeCommandesProduits`, le traitement dépend du type :

- **CommandeProduit**

1. Récupération de l'identifiant du stock correspondant :

```
String idStock = StockDAO.getIdStock(...);
```

2. Vérification stricte de la disponibilité du stock avec un `SELECT ... FOR UPDATE` pour gérer les récupérations en boutique qui vident directement le stock :

```
stockService.stock_suffisant_produit(...);
```

3. En cas d'insuffisance, la transaction est interrompue par une exception SQL.
4. Création et enregistrement d'une ligne de commande :

```
LigneCommande ligne = new LigneCommande(...);  
ligne.enregistrerLigneCommande(conn);
```

5. Mise à jour du poids total et du total de livraison.

- **CommandeContenant** Les étapes suivent la même logique, à l'exception de la gestion du stock si elle n'est pas nécessaire.

Justification du choix Repeatable Read

Le niveau d'isolation *Repeatable Read* est nécessaire pour ce type de logique métier, car :

- Il empêche un changement du stock entre plusieurs lectures.
- Il évite que des transactions concurrentes créent ou modifient des commandes affectant les mêmes produits.
- Il garantit que les données utilisées pour vérifier le stock sont strictement identiques tout au long de la transaction.

Dans un contexte où la fiabilité du stock est essentielle, ce choix est donc justifié.

Fonctionnement au niveau applicatif

Cette section présente la logique mise en place au niveau de l'application afin de préparer la transaction. Le traitement complet se déroule en deux grandes phases : la constitution du panier du client et l'exécution de la transaction de validation de la commande.

Construction du panier Lorsqu'un client sélectionne un produit ou un contenant, l'application exécute les étapes suivantes :

1. L'interface récupère toutes les informations saisies par le client (type, quantité, référence ou identifiant du produit).
2. Un objet métier est créé :
 - `CommandeProduit` si la commande concerne un produit,
 - `CommandeContenant` dans le cas d'un contenant.
3. Le produit ou contenant est disponible, il est ajouté au panier, représenté par :

`ArrayList<Commande> panier`

Choix du mode de récupération Avant même l'exécution de la transaction, le système demande au client de choisir un mode de récupération : retrait en boutique ou livraison à domicile.

Création de la commande finale Une fois le panier constitué et validé, l'application crée un objet représentant la commande complète :

```
CommandeFinale commande = new CommandeFinale(panier);
```

Cet objet contient :

- la liste totale des commandes individuelles (produits et contenants),
- un identifiant unique de commande généré automatiquement,
- les informations liées au mode de récupération.

Déclenchement de la transaction L'appel à :

```
commande.validerCommande();
```

déclenche alors l'exécution de la transaction SQL elle-même.

Cette séparation claire entre :

- la phase applicative (constitution et préparation du panier),
 - et la phase transactionnelle (vérification stricte + enregistrements SQL),
- permet d'assurer à la fois une bonne ergonomie côté utilisateur et un haut niveau de fiabilité au niveau du système de stockage.

Ainsi, l'application garantit que les produits proposés sont valides au moment de l'ajout au panier, puis revérifiés dans un environnement transactionnel isolé au moment de la validation finale.

Conclusion

La méthode `validerCommande()` assure une gestion fiable et cohérente de la validation d'une commande. Grâce à l'usage d'un niveau d'isolation assez strict et d'une logique transactionnelle complète, ce système permet d'éviter toute incohérence entre le stock, les lignes de commande et les informations associées.

Il garantit que l'ensemble du processus est atomique, isolé et cohérent, conformément aux exigences des transactions de type ACID. (voir figure 1)

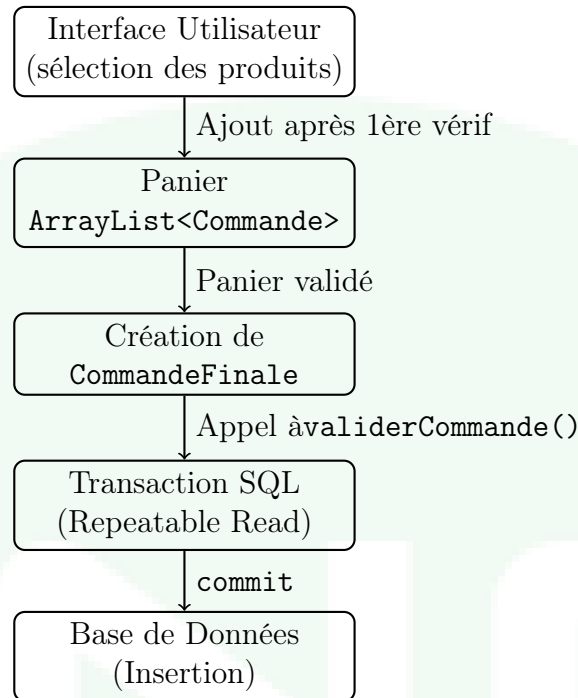


FIGURE 1 – Pipeline applicatif complet : de l’interface utilisateur à la transaction SQL.

5.2 Fonctionnalité 2 : Alertes de péremption et réduction des prix

Cette fonctionnalité permet de détecter les lots dont la date limite approche, d’informer les gérants, et d’appliquer automatiquement une réduction de prix afin de limiter les pertes et d’améliorer l’écoulement du stock.

Objectifs

Le système doit :

- identifier les lots dont la date limite survient dans les 7 jours ;
- afficher ces alertes dans l’interface ;
- appliquer une réduction de 30% sur le prix de vente des produits concernés ;
- mettre à jour automatiquement le statut du produit ;
- garantir la cohérence des données en cas d’accès concurrent.

Règles métier

- Une alerte est générée si la date limite vérifie :

$$0 < dateLimite - Aujourd'hui \leq 7$$

- La réduction s’applique sur le **prix de vente client** du conditionnement :

$$prix_nouveau = 0.7 \times prix_actuel$$

- La réduction concerne tous les conditionnements d’un même produit d’un producteur donné.
- Les produits déjà périmés ($dateLimite \leq \text{aujourd’hui}$) ne doivent pas apparaître dans les alertes.
- Les produits au statut **Pertes** sont exclus : aucune réduction ne peut leur être appliquée.
- Lorsqu’une réduction est appliquée avec succès, le champ *Statut* est mis à jour automatiquement :

$$Enattente \rightarrow Rductiond\grave{a}appliquer$$

- Le système ne limite pas le nombre de réductions possibles : un gestionnaire peut appliquer une réduction plusieurs fois si nécessaire. L’interface lui indique toutefois via le statut si une réduction a déjà été appliquée.

Gestion de la concurrence

L’application de la réduction s’effectue dans une transaction utilisant **FOR UPDATE** afin de verrouiller les lignes concernées et d’éviter toute écriture concurrente incohérente.

- Le premier gestionnaire obtenant le verrou peut appliquer la réduction et valider la transaction avec un **COMMIT**.
- Si un second gestionnaire tente d’appliquer la même réduction simultanément, sa transaction échoue : une erreur lui est renvoyée, indiquant que le produit a déjà été modifié entre-temps.
- L’interface doit alors afficher un message du type : “La réduction a déjà été appliquée par un autre utilisateur.”

Intégration dans l’interface

L’interface graphique Swing :

- récupère la liste des alertes via **AlertePeremptionService** ;
- affiche le nom du produit, la date limite, les jours restants, le statut actuel et la réduction proposée ;
- permet d’appliquer et de confirmer la réduction en un clic ;
- met à jour en temps réel la colonne *Statut* pour refléter l’application de la réduction ;

- gère les erreurs de concurrence en affichant un message explicite en cas de modification simultanée.

Résumé

Cette fonctionnalité offre :

- une détection fiable des produits proches de la péremption ;
- une réduction automatique conforme aux règles métier ;
- une mise à jour du statut permettant de suivre facilement les actions déjà réalisées ;
- une gestion robuste de la concurrence grâce aux transactions SQL ;
- une intégration fluide et informative dans l'interface utilisateur.



5.3 Fonctionnalité 3 : Clôture d'une commande (retrait ou livraison)

Cette section présente la mise en place d'une transaction SQL permettant de garantir la cohérence lors de la clôture d'une commande, que ce soit pour un retrait en boutique ou une livraison à domicile.

Les méthodes `preparerCommande()`, `cloturerCommande()` et `marquerPrete()` implémentent l'ensemble de cette logique transactionnelle et constituent le cœur de cette fonctionnalité.

Objectifs

La transaction doit garantir les éléments suivants :

- Mettre à jour le statut de la commande selon le mode de récupération (Boutique ou Domicile)
- Gérer le paiement selon le mode choisi (En ligne ou En Boutique)
- Si livraison : calculer et afficher les frais de livraison finaux et la date estimée
- Gérer le stock selon la stratégie FEFO (First-Expired, First-Out) et le mode de paiement
- Enregistrer la date effective de récupération/livraison
- Respecter les règles métier spécifiques à chaque mode de récupération

Règles métier

La clôture d'une commande suit des règles différentes selon le mode de récupération :

- **Commandes Domicile :**
 - Paiement obligatoire "En ligne" (vérifié avant la préparation)
 - Statut initial : "En préparation"
 - Passage à "Prête" : déclenche la sortie de stock et l'enregistrement de la date de paiement
 - Clôture : enregistre la date de récupération/livraison et passe le statut à "Récupérée/Livrée"
- **Commandes Boutique :**
 - Pas de statut "En préparation" (statut passe directement à "Prête" à la création)
 - Paiement "En Boutique" possible uniquement si le statut est "Prête"
 - Sortie de stock déjà effectuée lors de la création de la commande
 - Clôture : enregistre le paiement si nécessaire, enregistre la date de récupération et passe le statut à "Récupérée/Livrée"
- **Contraintes générales :**

- Clôture interdite si la commande est déjà clôturée (dateRecuperation non nulle ou statut final)
- Le statut doit être “Prête” pour pouvoir clôturer

Calcul des frais de livraison Pour les commandes avec mode de récupération “Domicile”, nous avons décidé de calculer les frais de livraison selon la façon suivante :

- **Frais de distance :**
 - 0-50 km : 0€
 - 50-300 km : 1€
 - 300-2000 km : 2€
 - >2000 km : 3€
- **Frais supplémentaire liés au poids :**
 - France Métropolitaine : 0.8€ par kilo
 - DOM-TOM : 1.2€ par kilo
 - International : 1.5€ par kilo
- **Total :** arrondi à l’entier supérieur

Le calcul est effectué via :

```
int frais = commandeDAO.calculFraisDeLivraison(idModeRecuperationDomicile);
```

Calcul de la date estimée de livraison La date estimée de livraison est calculée en fonction de :

- **Jours de base selon la zone :**
 - France Métropolitaine : 2 jours
 - DOM-TOM : 5 jours
 - International : 7 jours
- **Jours supplémentaires selon la distance :**
 - 0-50 km : +0 jour
 - 50-300 km : +1 jour
 - 300-2000 km : +2 jours
 - >2000 km : +3 jours
- **Délai maximal de disponibilité :** ajout du délai maximal de disponibilité des produits commandés (temps d’acquisition par l’épicerie)

Le calcul est effectué via :

```
LocalDate dateEstimee = commandeDAO.calculDateEstimeeDeLivraison(idModeRecuperationDomicile);
```

Justification du choix **SERIALIZABLE**

Le niveau d'isolation *SERIALIZABLE* est nécessaire pour ce type de logique métier, car :

- Il empêche un changement du stock entre plusieurs lectures.
- Il évite que des transactions concurrentes modifient le statut ou les informations d'une commande pendant sa clôture.
- Il garantit que les données utilisées pour vérifier le statut et le stock sont strictement identiques tout au long de la transaction.
- Il prévient les problèmes de concurrence lors du calcul des frais de livraison et de la date estimée.

Oracle ne supportant que "READ COMMITTED" et "SERIALIZABLE", le choix de "SERIALIZABLE" permet d'obtenir le niveau d'isolation le moins strict et plus sécurisé disponible.

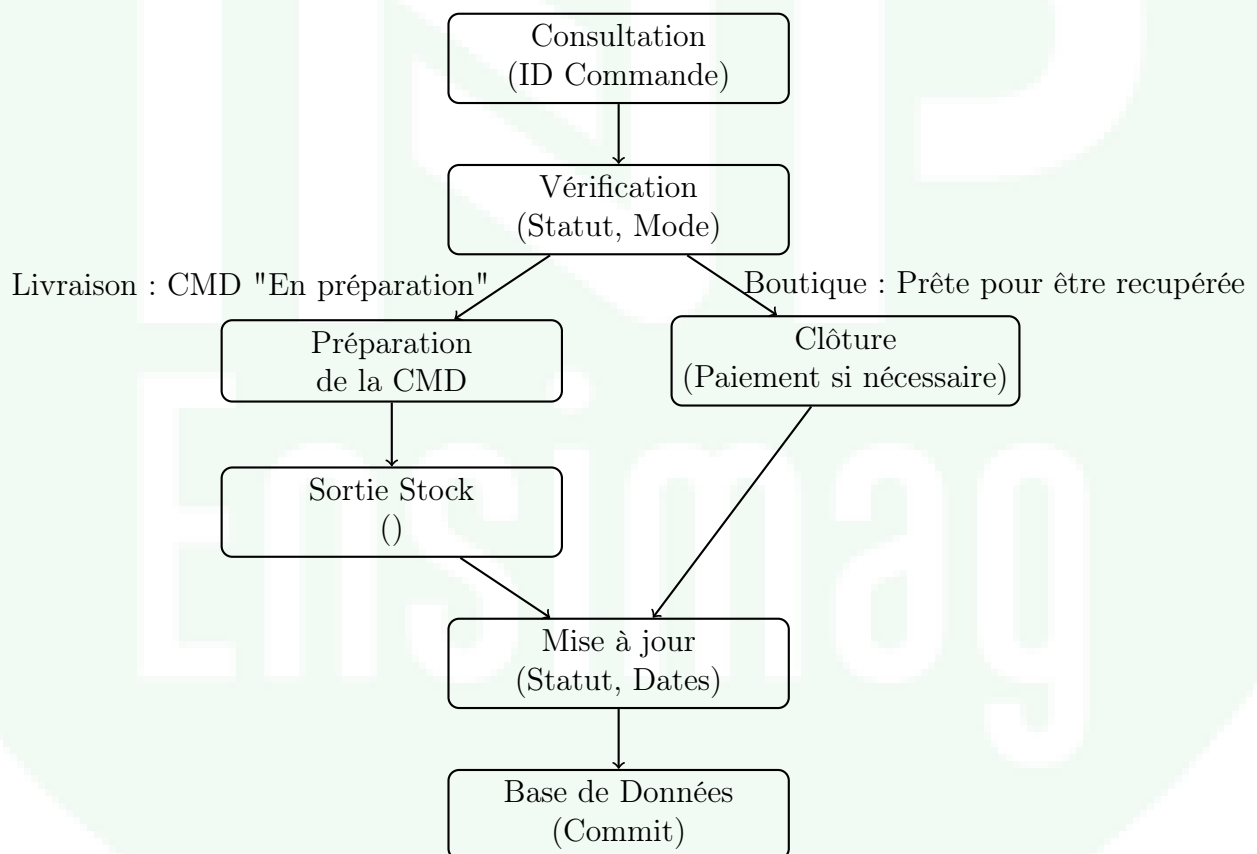


FIGURE 2 – Processus de clôture d'une commande : de la consultation à la finalisation.

6 BILAN DU PROJET

Le projet a été mené à terme en cinq semaines, grâce à une organisation rigoureuse et une division des tâches qui a permis un avancement efficace.

6.1 Organisation de l'Équipe et Déroulement

Nous avons opté pour une approche par tâches successives pour garantir la cohérence des étapes fondamentales du projet.

- **Modélisation Collective (Entités/Associations et Schéma Relationnel) :** L'équipe a pris la décision de réaliser l'étape cruciale de la modélisation (analyse du problème, Entités/Associations, traduction relationnelle) intégralement en équipe. Ce travail collaboratif a été essentiel pour s'assurer que tous les membres partageaient une compréhension profonde et unifiée des besoins complexes du client (saisonnalité, stock en lots, conditionnement, etc.).
- **Division des Tâches (Implantation et Fonctionnalités) :** Une fois le schéma relationnel validé, nous avons divisé l'implantation de la base de données (scripts SQL de création et de peuplement) et le développement de l'application Java. Chaque membre s'est vu attribuer des fonctionnalités spécifiques à implémenter (Passage de commande, Alertes de péremption, Clôture de commande).

6.2 Outils et Choix Techniques

- **Gestion de Version :** Nous avons utilisé GitLab pour le travail collaboratif sur le code source. Cet outil en ligne a facilité la gestion des versions, le partage des scripts SQL et du code Java, et la résolution des conflits, assurant une intégration continue et fluide du travail de chacun.
- **Implémentation Java :** Pour structurer le code Java du démonstrateur, nous avons adopté la logique DAO (Data Access Object). Ce choix a permis de séparer clairement la logique métier de l'accès à la base de données via JDBC, améliorant la modularité, la lisibilité et la maintenabilité du code, même si l'évaluation se concentre uniquement sur les aspects Base de Données.

6.3 Difficultés Rencontrées

La réalisation de ce projet a soulevé plusieurs défis significatifs que l'équipe a dû surmonter.

- **Complexité et Temps Alloué à la Modélisation et Conception :** La phase initiale d'analyse du problème et de conception de la base de données

(schéma Entités/Associations et traduction relationnelle) a nécessité un investissement en temps considérable pour l'équipe entière. Le travail collectif sur cette étape était nécessaire pour appréhender des contraintes métier fines et complexes comme la gestion des deux modes de conditionnement (vrac/pré-conditionné) , la modélisation de la saisonnalité , et la gestion spécifique des stocks par lots (DLC/DLUO). Ce temps passé ensemble a été un gage de qualité, mais a engendré un léger retard sur le planning initial.

— **Difficultés d'Implantation sur l'Environnement Oracle :**

La traduction du schéma relationnel et son implantation concrète sur le SGBD Oracle se sont révélées plus ardues que prévu. Nous avons rencontré des difficultés techniques (syntaxe, contraintes d'intégrité complexes, types de données) lors de la création du schéma relationnel et de l'insertion des jeux de données pertinents.

— **Nécessité de Rétro-Ingénierie et Ajustements de Modèle en Cours de Projet :**

La difficulté majeure a été la découverte d'erreurs ou d'imprécisions dans notre modélisation initiale au moment de l'écriture du code Java/JDBC et de l'implémentation des requêtes SQL pour les transactions. Par exemple, la logique précise de la stratégie de gestion du stock pendant le cycle de vie d'une commande ou le calcul exact de date de livraison ont révélé des lacunes dans les relations ou les attributs choisis. Ceci a nécessité de revenir en arrière sur les étapes de modélisation et de conception pour modifier le schéma relationnel, les scripts SQL, et par conséquent, le code Java déjà écrit, ralentissant le déroulement global du projet.

Malgré ces obstacles, le choix d'un travail collectif sur la modélisation initiale a permis de mieux absorber les modifications nécessaires et d'assurer un modèle final beaucoup plus stable et conforme aux exigences fonctionnelles.

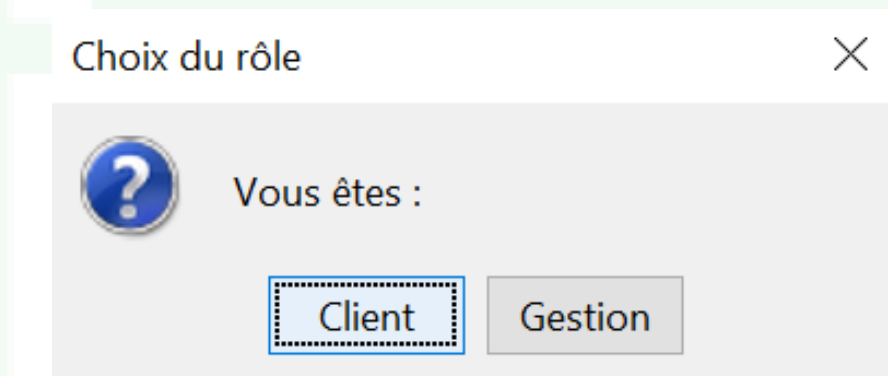
7 GUIDE D'UTILISATION@

7.1 Lancer l'application :

Commande make : Il suffit d'exécuter la commande **make** à la racine du projet (pas dans le dossier **src/**) pour lancer la fenêtre de connexion client/epicerie.

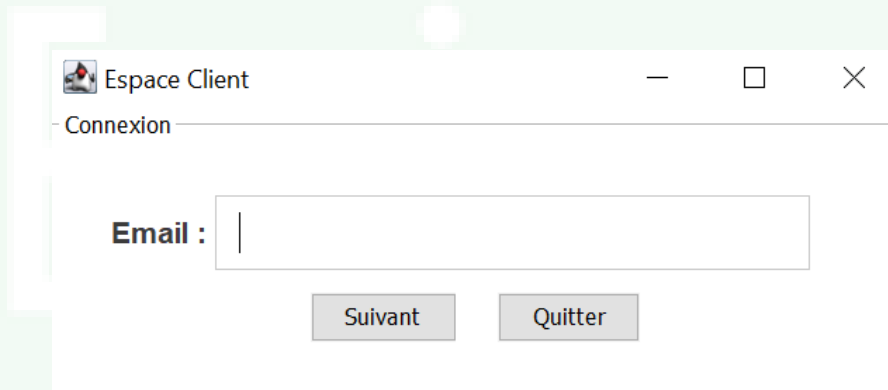
7.2 Fenêtre de sélection du rôle :

Choisir son rôle parmi les deux proposés et cliquer :



7.3 Login

On rentre donc à l'Espace Client. ex : luc.petit@email.com



Une fois l'email vérifié dans notre base de données, le client peut accéder à la Boutique.

7.4 Boutique : Epicerie au Valaisan

Après avoir cliqué, la boutique s'ouvre et vous pouvez choisir parmi les différentes options :



7.5 Catalogue des produits :

En cliquant sur le catalogue, la fenêtre correspondante s'ouvre :

Catalogue des Produits - Épicerie Locale

Filtres et Recherche

Catégorie: Rechercher:

Produits Disponibles

Vin Blanc Bio 2023	Boissons	14.00	Préconditionné	0,75 kg	Oui	Rhône	Disponible
Vin Rouge 2022	Boissons	12.50	Préconditionné	0,75 kg	Non	Rhône	Disponible
Farine de Blé Bio	Céréales	0.20	Vrac	N/A	Oui	Isère	Disponible
riz arabe	Céréales	100.00	N/A	N/A	Oui	Morocco	Disponible
Chèvre Fermier	Fromage	12.00	Préconditionné	0,25 kg	Non	Rhône-Alpes	Disponible
Tomme Fermière	Fromage	15.00	Préconditionné	0,50 kg	Oui	Rhône-Alpes	Disponible
Carottes Bio	Légumes	3.50	Préconditionné	1,00 kg	Oui	Isère	Disponible
Mesclun Bio	Légumes	4.00	Préconditionné	0,20 kg	Oui	Isère	Disponible
Tomates Anciennes	Légumes	5.50	Préconditionné	0,50 kg	Oui	Isère	Disponible
Miel d'Acacia	Miel	15.50	Vrac	N/A	Oui	Isère	Disponible
Miel de Montagne	Miel	18.00	Vrac	N/A	Oui	Isère	Disponible

Pour voir les détails d'un produit, il suffit de cliquer sur ce produit puis sur "Voir les détails" :

Catalogue des Produits - Épicerie Locale

Filtres et Recherche

Catégorie: Toutes Rechercher Réinitialiser

Produits Disponibles

Vin Blanc Bio 2023	Boissons	14,00	Préconditionné	0,75 kg	Oui	Rhône	Disponible
Vin Rouge 2022	Boissons	12,50	Préconditionné	0,75 kg	Non	Rhône	Disponible
Farine de Blé Bio	Céréales	0,20	Vrac	N/A	Oui	Isère	Disponible
riz arabe	Céréales	100,00	N/A	N/A	Oui	Morocco	Disponible
Chèvre Fermier	Fromage	12,00	Préconditionné	0,25 kg	Non	Rhône-Alpes	Disponible
Tomme Fermière	Fromage	15,00	Préconditionné	0,50 kg	Oui	Rhône-Alpes	Disponible
Carottes Bio	Légumes	3,50	Préconditionné	1,00 kg	Oui	Isère	Disponible
Mesclun Bio	Légumes	4,00	Préconditionné	0,20 kg	Oui	Isère	Disponible
Tomates Anciennes	Légumes	5,50	Préconditionné	0,50 kg	Oui	Isère	Disponible
Miel d'Acacia	Miel	15,50	Vrac	N/A	Oui	Isère	Disponible
Miel de Montagne	Miel	18,00	Vrac	N/A	Oui	Isère	Disponible

Voir Détails Retour Quitter

On peut alors visualiser les détails du produit :

Détails du Produit



Vin Blanc Bio 2023

Catégorie: Boissons
Prix: 14,00 €
Type: Préconditionné
Poids/Unité: 0,75 kg
Bio: Oui
Origine: Rhône
Statut: **Disponible**

OK

Après avoir consulté le catalogue, retourner à la boutique pour passer commande.

7.6 Passer une commande :

Pour passer une commande : Cliquer sur l'option "Passer Commande" :

Espace Client

Connexion

Email :

Suivant Quitter

Une fois l'email est vérifié dans notre base de données, le client peut accéder à la fenêtre de commande.

Un email valide est le suivant : **luc.petit@email.com**

La fenêtre de commande se lance automatiquement :

The screenshot shows a web application window titled "Passer une commande". It is divided into several sections:

- Sélection du produit et contenants:** This section contains two dropdown menus. The first, labeled "Produit :", has the selected value "Vin Blanc Bio 2023 - 14.0 EUR (0.75 kg)". Below it is a green button labeled "Ajouter au panier". The second dropdown, labeled "Contenant :", has the selected value "Bocal en verre réutilisable (0.5L) - 2,50 EUR - Réutilisable". Below it is a text input field for "Quantité :" with the value "1" and a small up/down arrow. A blue button labeled "Ajouter contenant au panier" is positioned below the quantity field.
- Votre panier:** This section features a table with the following headers: "Produit", "Type", "Prix unitaire", "Quantité", and "Sous-total". The table body is currently empty. Below the table, it displays "Total : 0.00 EUR" and a red button labeled "Vider le panier".
- Mode de paiement et récupération:** At the bottom, there are two more dropdown menus. "Mode de paiement :" is set to "En ligne", and "Mode de récupération :" is set to "Boutique".
- Action buttons:** At the very bottom, there are three buttons: "Valider la commande" (blue), "Retour" (blue), and "Quitter" (red).

On peut alors : choisir parmi les produits et contenants → sélectionner la quantité → ajouter au panier/vider le panier → choisir le mode de paiement et le mode de récupération parmi les options proposées → valider la commande.

Commande invalide :

Passer une commande

Sélection du produit et contenants

Produit : Farine de Blé Bio - 0.2 EUR/kg (Vrac)

Ajouter au panier

Contenant : Bocal en verre réutilisable (0.5L) - 2,50 EUR - Réutilisable

Quantité : 1

Ajouter contenant au panier

Erreur

Erreur lors de la création de la commande :
Stock insuffisant pour le produit: PD1111111

OK

Votre panier

Produit	Type	Sous-total
Farine de Blé Bio	Preconditionne	0,20 EUR

Total : 0,20 EUR

Vider le panier

Mode de paiement : En ligne

Mode de récupération : Boutique

Valider la commande Retour Quitter

Commande valide :

Passer une commande

Sélection du produit et contenants

Produit : Vin Blanc Bio 2023 - 14.0 EUR (0.75 kg)

Ajouter au panier

Contenant : Bocal en verre réutilisable (0.5L) - 2,50 EUR - Réutilisable

Quantité : 1

Ajouter contenant au panier

Succès

Commande validée avec succès !
Numéro de commande : CM6626970

OK

Votre panier

Produit	Type	Sous-total
Vin Blanc Bio 2023	Preconditionne	14,00 EUR

Total : 14,00 EUR

Vider le panier

Mode de paiement : En ligne

Mode de récupération : Boutique

Valider la commande Retour Quitter

Quand la commande est validée, vous revenez à la boutique pour finaliser la commande.

7.7 Alertes de Prémption :

L'Epicier peut consulter la table de prémption et appliquer la réduction :

Alertes de Prémption
1 alerte(s) trouvée(s)

Produits proches de la péremption (< 7 jours)

Produit	Fournisseur	Nom	Quantité	Date d'expiration	Réduction	Urgence	
LT1111110	PD1111111	PR1234567	Farine de Blé Bio	1 jour(s)	2025-11-30	30%	URGENT

Actualiser Appliquer Réduction Retour Quitter

Cliquer sur le produit souhaité, puis appliquer la réduction, et enfin revenir en boutique pour voir le nouveau prix.

7.8 Clôturer la commande :

Cliquer sur la case "Clôture de commande" ouvre la fenêtre suivante :

Clôture de Commande
Finaliser le retrait en boutique ou la livraison à domicile

Informations de la Commande

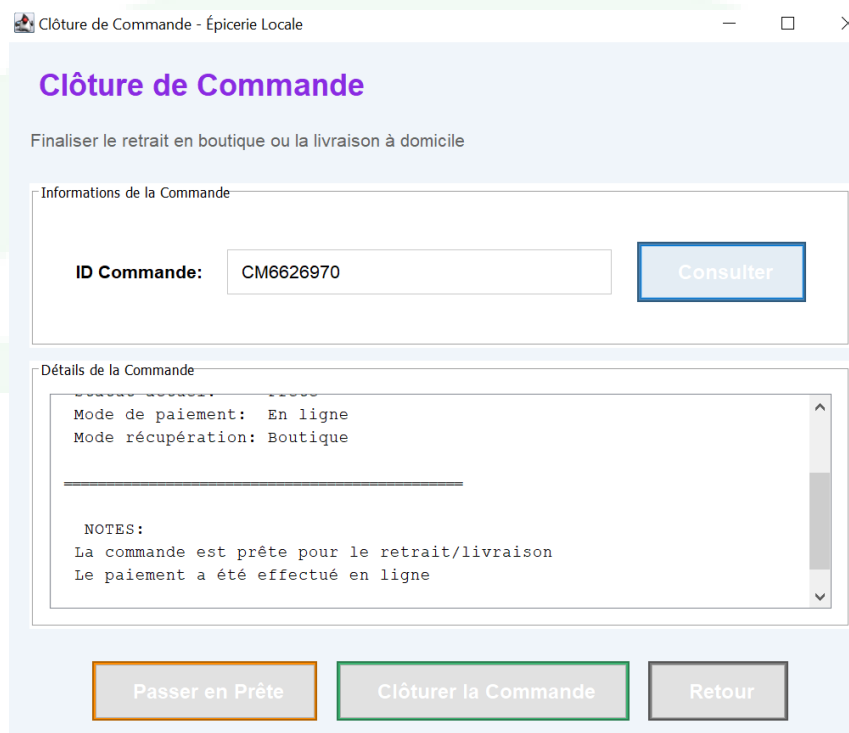
ID Commande: Consulter

Détails de la Commande

Veuillez saisir un ID de commande et cliquer sur 'Consulter'

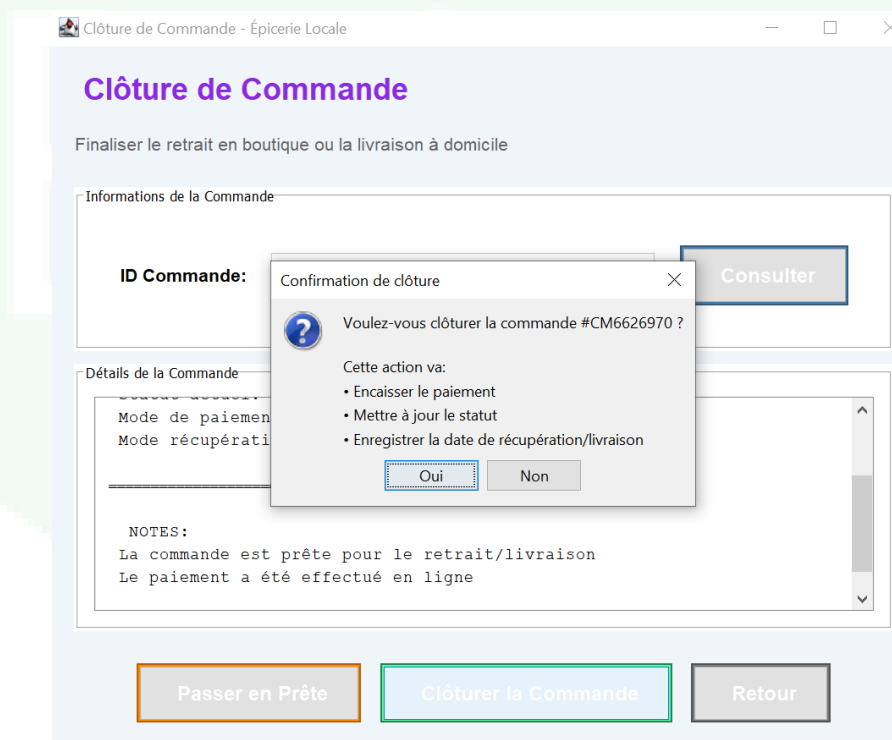
Passer en Prête Clôturer la Commande Retour

La phase de clôture dépend de l'ID de commande que le client a reçu en validant son panier et il pourra consulter les détails de sa commande.



The screenshot shows a web application window titled "Clôture de Commande - Épicerie Locale". The main heading is "Clôture de Commande" in purple. Below it, the instruction "Finaliser le retrait en boutique ou la livraison à domicile" is displayed. The "Informations de la Commande" section contains the "ID Commande: CM6626970" and a "Consulter" button. The "Détails de la Commande" section shows "Mode de paiement: En ligne" and "Mode récupération: Boutique". A "NOTES:" section contains the text: "La commande est prête pour le retrait/livraison" and "Le paiement a été effectué en ligne". At the bottom, there are three buttons: "Passer en Prête" (orange border), "Clôturer la Commande" (green border), and "Retour" (grey).

En cliquant sur "Clôturer la commande" :



This screenshot shows the same "Clôture de Commande" window, but with a confirmation dialog box overlaid in the center. The dialog box is titled "Confirmation de clôture" and contains a question mark icon, the text "Voulez-vous clôturer la commande #CM6626970 ?", and a list of actions: "Encaisser le paiement", "Mettre à jour le statut", and "Enregistrer la date de récupération/livraison". At the bottom of the dialog are "Oui" and "Non" buttons. The "Clôturer la Commande" button in the background is now highlighted with a cyan border.

Exemple de clôture valide de la commande :

Clôture de Commande - Épicerie Locale

Clôture de Commande

Finaliser le retrait en boutique ou la livraison à domicile

Informations de la Commande

ID Commande: CM6626970 Consulter

Détails de la Commande

Mode de paiement:
Mode récupération:

NOTES:
La commande est prête pour le retrait/livraison
Le paiement a été effectué en ligne

Passer en Prête Clôturer la Commande Retour

Succès

✓ Commande clôturée avec succès !
La commande #CM6626970 a été finalisée.
OK

NB : clôturer une commande veut dire qu'on diminue le stock automatiquement

8 CONCLUSION

Ce projet a été une excellente opportunité pour notre équipe de mettre en pratique et d'intégrer nos compétences en systèmes de gestion de bases de données relationnelles et en programmation d'applications transactionnelles. Nous avons non seulement réussi à livrer un schéma de base de données complet et cohérent pour l'épicerie "Au Valaisan", mais aussi à développer un démonstrateur Java/JDBC fonctionnel qui implémente les fonctionnalités critiques définies. Les défis liés à la gestion du stock par lots (DLC/DLUO), à la saisonnalité, et à la gestion de la concurrence des commandes ont été résolus de manière satisfaisante. L'expérience d'une collaboration structurée, en particulier sur la modélisation et l'utilisation d'outils professionnels comme GitHub et la logique DAO, a été très formatrice. Nous sommes convaincus que notre solution répond de manière complète et robuste aux besoins du client, tout en respectant les contraintes techniques et légales (droit à l'oubli, conservation des historiques).